

Finlay - 6880463 AO2 Financial Tracker

Below is a list of all the project requirements

- At the start of the program, ask the user if they want to **resume a previous session** by reading the transaction log (logs/transaction_log.txt). - COMPLETED
- If resuming, read the previous transactions from the file and store them in a **linked list**. - COMPLETED
- If no log file exists, start with an empty linked list. - COMPLETED
- The program must maintain all transactions dynamically using a **linked list**. - COMPLETED
- Users can **insert** a new transaction at any position (beginning, middle, or end). - COMPLETED
- Users can **delete** any specific transaction by its position in the list. - COMPLETED
- Transactions should not be immediately written to the file—only updated in the **linked list** while the program is running. - COMPLETED
- The program should provide the following commands: - COMPLETED
 1. **add income** – Add an income transaction (with a description and amount). After adding, displays whether the user is within budget or in debt.
 2. **add expense** – Add an expense transaction (with a description and amount). After adding, displays whether the user is within budget or in debt.
 3. **delete <position>** – Remove a transaction from the linked list (position is based on list order).
 4. **print** – Display all transactions in the linked list.
 5. **quit** – Save all transactions to `logs/transaction\_log.txt` and exit the program.
- A menu-driven interface, where the user is presented with a list of options and selects one by entering its corresponding number. - COMPLETED
- **Transactions should only be written to the log file when the user quits.** - COMPLETED
- If the program stops unexpectedly, the log file should **remain unchanged**. - COMPLETED
- When quitting, **overwrite the existing log file** with the latest linked list data. - COMPLETED
- The print command should display transactions with symbols indicating modifications: - COMPLETED
 - **+++ i** → Newly inserted transactions (not yet saved to the file). - COMPLETED
(I was not entirely sure what was meant by this, so I wrote the code so that transactions inserted into the middle of the linked list had the identifier '+++ i' rather than 'new')
 - **--- d** → Transactions marked for deletion (removed from the linked list but not yet from the file). - INCOMPLETE
(I did not understand this section at all, the method I ended up using is definitely incorrect, as it is for expenses that have been inserted in the middle, but once I realised it was incorrect I didn't know what to replace it with)

(the delete command is meant to "Remove a transaction from the linked list", but the ---d signifier is for "Transactions marked for deletion (removed from the linked list but not yet from the file))

(in which case, where are you meant to be reading the transaction from? the file will not have updated at this point so it would be the same, but it no longer exists in the linked list)

(I considered an approach where I overwrite its information within the linked list to be blank before you exit as a kind of intermediary phase, but then what's even the point as there is no way to get it back anyways?)

- Ensure a **clear and structured output format** (see sample below). - COMPLETED
- **Reject negative values** for income and expenses. - COMPLETED
- **Reject non-numeric inputs** for amounts. - COMPLETED
- **Handle invalid delete positions gracefully** (e.g., deleting an out-of-range entry should not crash the program). - COMPLETED
- Compiles all .c files and generates an executable named **fintrack** in the release/ directory. COMPLETED
- Supports make clean to remove all .o files and the executable. - COMPLETED
- Supports `make dist` to create a ZIP archive (dist/fintrack.zip) containing src/, include/, logs/transaction_log.txt, report/, and Makefile. - COMPLETED
- Running `make` without arguments should execute clean, compile the executable, and create the distribution ZIP. - COMPLETED
- **Split code into multiple files** (.c and .h files) and organize them into folders (src/, include/).- COMPLETED
- Use **descriptive function and variable names**.- COMPLETED
- Include **comments** explaining the purpose of functions and structures. COMPLETED
- Ensure the program is **modular and professional**—this will be part of your grade.- COMPLETED

My approach to this code was to make a switch case menu which runs the functions for each corresponding command.

I split my work into 3 main files:

- main - which is responsible for the main menu and the cleanup. It includes 4 functions

main() - asks the user if they want to continue the previous session and contains the primary switchcase/main menu loop

```
//main function
int main(void){
    char resumePreviousSession[256];
    char choice[10];

    printf("Welcome to your Personal Finance Tracker!\n");
    printf("Would you like to resume your previous session? (y/n): ");

    fgets(resumePreviousSession, sizeof(resumePreviousSession), stdin);

    if (resumePreviousSession[0] == 'y' || resumePreviousSession[0] == 'Y') {
        printf("Resuming from last session...\n");
        load_transactions();
    } else {
        printf("Starting a new session...\n");
    }
}
```

```

//main menu loop
while (1) {
    display_menu();
    fgets(choice, sizeof(choice), stdin);

    switch(choice[0]) {
        case '1':
            add_income();
            break;
        case '2':
            add_expense();
            break;
        case '3':
            delete_transaction();
            break;
        case '4':
            print_transactions();
            break;
        case '5':
            cleanup();
            printf("Done. Exiting program.\n");
            return 0;
        default:
            printf("Invalid option. Please try again.\n");
    }
}

return 0;
exit(0);
}

```

display_menu() - prints the options available to the user

```

//function to display menu
void display_menu(void) {
    double balance = get_total_money();

    printf("\n--- Personal Finance Tracker Menu ---\n");
    printf("1. Add Income\n");
    printf("2. Add Expense\n");
    printf("3. Delete\n");
    printf("4. Print\n");
    printf("5. Exit\n");
    printf("-----\n");
    printf("Current balance: $%.2f\n", balance);
    if (balance < 0.0) {
        printf("Budget status: Over budget\n");
    } else{
        printf("Budget status: Within budget\n");
    }
    printf("Select an option (1-5): ");
}

```

get_total_money() - goes through the linked list node by node and adds up the total money

```
double get_total_money(void) {  
    double total = 0.0;  
    TransactionNode *current = head;  
  
    while (current != NULL) {  
        total += current->amount;  
        current = current->next;  
    }  
  
    return total;  
}
```

cleanup() - changes the status of each node to saved, saves it to the file and then clears the linked list

```
//function to save contents of the linked list to file and free all memory
void cleanup(void) {
    TransactionNode *current = head;
    while (current != NULL) {
        strcpy(current->status, "saved");
        current = current->next;
    }

    save_transactions();

    current = head;
    while (current != NULL) {
        TransactionNode *temp = current;
        current = current->next;
        free(temp);
    }
    head = NULL;
}
```

- fileHandler - which is responsible for loading to and from the transaction log file, it includes 2 functions

save_transactions() - checks if the file exists, if it does, it overwrites the entire file and goes node by node and writes the contents of it on each line

```
void save_transactions(void) {
    FILE *file = fopen(LOG_FILE, "w");
    if (file == NULL) {
        printf("Error: Could not open file for writing\n");
        return;
    }

    TransactionNode *current = head;
    while (current != NULL) {
        fprintf(file, "%s|%.2f|s\n", current->description, current->amount, current->status);
        current = current->next;
    }

    fclose(file);
    printf("Transactions saved successfully!\n");
}
```

load_transactions() - checks if the file exists, making one if it doesn't, then it checks each line for up to 99 characters followed by a '|' then a floating point number followed by another

'|'and then either 99 more characters or the end of the line, if it finds a line of this description, it loads each of the three into variables and then to a new node of the linked list, then it moves on to the next line

```
void load_transactions(void) {
    FILE *file = fopen(LOG_FILE, "r");
    if (file == NULL) {
        printf("No previous transactions found. Starting program.\n");
        return;
    }

    char line[300];
    char description[100], status[20];
    double amount;

    while (fgets(line, sizeof(line), file) != NULL) {
        line[strcspn(line, "\n")] = 0;

        //scans each line for anything fitting the format of some text|some number|some text
        if (sscanf(line, "%99[^\n]|%1f|%99[^\n]", description, &amount, status) == 3) {
            TransactionNode *new_node = create_transaction_node(description, amount, status);
            if (new_node != NULL) {
                insert_node_at_position(new_node, count_transactions() + 1);
            }
        }
    }

    fclose(file);
    printf("Previous transactions loaded.\n");
}
```

- linkedListHandler - which is responsible for adding, removing and printing the list of transactions logically in the linked list, it has 7 functions

create_transaction_node() - creates a new node in the data structure, checks if its null and then loads the description, amount, status and then sets the next node to null

```
// function for creating a new transaction node
TransactionNode *create_transaction_node(const char *description, double amount, const char *status) {
    TransactionNode *new_node = (TransactionNode *)malloc(sizeof(TransactionNode));
    if (new_node == NULL) {
        printf("Error: Memory allocation failed\n");
        return NULL;
    }

    strcpy(new_node->description, description);
    new_node->amount = amount;
    strcpy(new_node->status, status);
    new_node->next = NULL;

    return new_node;
}
```

count_transactions() - makes an int called count and then iterates through the linked list until it finds a null node and then returns the count

```
//function to count total transactions
int count_transactions(void) {
    int count = 0;
    TransactionNode *current = head;
    while (current != NULL) {
        count = count + 1;
        current = current->next;
    }
    return count;
}
```

insert_node_at_position() - checks if the node is null, then if the node is being set inserted at position 1, it sets the current head to the next node and makes the new node the new head. if its not at position 1, it increases the count and makes a new node after the position and changes the links before and after to apply to the new node

```

//function to insert a transaction node at a specific position
void insert_node_at_position(TransactionNode *new_node, int position) {
    if (new_node == NULL) {
        return;
    }

    if (position == 1) {
        new_node->next = head;
        head = new_node;
        return;
    }else{
        TransactionNode *current = head;
        int count = 1;
        while (current != NULL && count < position - 1) {
            current = current->next;
            count = count + 1;
        }
        if (current != NULL) {
            new_node->next = current->next;
            current->next = new_node;
        }
    }
}

```

print_transactions() - checks if the list is empty and prints that its empty if it is, thne it goes through the linked list and prints each value in the format, iterating the count as it does so it can print the transaction number as well

```

//function to print transactions
void print_transactions(void) {
    if (head == NULL) {
        printf("\nNo transactions to display\n");
        return;
    }

    printf("\n--- All Transactions ---\n");
    printf("Description          | Amount      | Status\n");
    printf("-----\n");

    TransactionNode *current = head;
    int count = 1;

    while (current != NULL) {
        printf("%d. %-30s | $%-9.2f | %s\n",
               count,
               current->description,
               current->amount,
               current->status);
        current = current->next;
        count = count + 1;
    }
    printf("-----\n");
}

```

delete_transaction() - checks if the list is empty, and then calls the print transaction function so that the user can see what they want to delete, then if the position for deletion is 1, it allocates the next node as the new head, and deletes the current head, otherwise it travels through the linked list until it finds the correct position and deletes it, relinking the node before and after it to connect to each other

```

void delete_transaction(void) {
    if (head == NULL) {
        printf("\nNo transactions to delete\n");
        return;
    }

    print_transactions();

    printf("\nEnter transaction number to delete: ");
    int index;
    scanf("%d", &index);
    getchar();

    if (index < 1) {
        printf("Error: Invalid transaction number\n");
        return;
    }

    if (index == 1) {
        TransactionNode *temp = head;
        head = head->next;
        free(temp);
        printf("Transaction deleted successfully!\n");
        return;
    }

    TransactionNode *current = head;
    TransactionNode *previous = NULL;
    int count = 1;

    while (current != NULL && count < index) {
        previous = current;
        current = current->next;
        count = count + 1;
    }

    if (current == NULL) {
        printf("Error: Invalid transaction number\n");
        return;
    }
}

```

add_income() - takes the users input for description and amount and position (making sure all are valid) then it checks if the new node would be at the end, if it is, it gives it the status 'new', if not it gets the status '+++i' then it makes the new node in the position with all the data

```
//function to add income
void add_income(void) {
    printf("\n--- Add Income ---\n");

    char description[100];
    double amount;
    int position;
    int total = count_transactions();

    printf("Description: ");
    fgets(description, sizeof(description), stdin);
    description[strcspn(description, "\n")] = 0;

    printf("Amount: ");
    scanf("%lf", &amount);
    getchar();//clear newline from input buffer, this took me ages to figure out

    if (amount <= 0) {
        printf("Error: Income amount must be positive!\n");
        return;
    }

    if (total > 0) {
        print_transactions();
        printf("\nEnter position to insert (1-%d or %d to add at end): ", total + 1, total + 1);
        scanf("%d", &position);
        getchar();

        if (position < 1 || position > total + 1) {
            printf("Error: Invalid position!\n");
            return;
        }
    } else {
        position = 1;
    }

    char *status;
    if (position == total + 1) {
        status = "new";
    } else {
        status = "+++ i";
    }

    TransactionNode *new_node = create_transaction_node(description, amount, status);
    if (new_node != NULL) {
        insert_node_at_position(new_node, position);
        printf("Income added successfully\n");
    }
}
```

add_expense() - same as above but with '---d' for expenses and converts to negative. I have since discovered that this was not what you intended, but I wasn't really sure what to change it with as I dont understand your instruction

```

//function to add expense
void add_expense(void) {
    printf("\n--- Add Expense ---\n");

    char description[100];
    double amount;
    int position;
    int total = count_transactions();

    printf("Description: ");
    fgets(description, sizeof(description), stdin);
    description[strcspn(description, "\n")] = 0;

    printf("Amount: ");
    scanf("%lf", &amount);
    getchar();

    if (amount <= 0) {
        printf("Error: Expense amount must be positive!\n");
        return;
    }

    if (total > 0) {
        print_transactions();
        printf("\nEnter position to insert (1-%d): ", total + 1);
        scanf("%d", &position);
        getchar();

        if (position < 1 || position > total + 1) {
            printf("Error: Invalid position!\n");
            return;
        }
    } else {
        position = 1;
    }

    char *status;
    if (position == total + 1) {
        status = "new";
    } else {
        status = "--- d";
    }

    amount = -amount;

    TransactionNode *new_node = create_transaction_node(description, amount, status);
    if (new_node != NULL) {
        insert_node_at_position(new_node, position);
        printf("Expense added successfully\n");
    }
}

```

I mostly used the built in debugger to locate issues within my code, but honestly, the majority of my code ran relatively seamlessly.

The major bugs which caused me grief were larger misunderstandings about how makefiles work, but various youtube videos helped me understand them.

I did struggle for a while due to not realising I had to clear the newline from the input buffer and also with the format of initialising structs in header files, but I figured both of those out

Overall, I think this project went well, other than my time management.

The one known issue is the '---d' status for files which have been deleted from the linked list but not from the file.

I don't know when that status would ever appear as the struct either exists in the linked list or it doesn't and the file hasn't been updated yet, so there wouldn't be a window for it to both exist but be marked for deletion that the user could see.

```

@finlaydes →/workspaces/A02_financial_tracker (main) $ make
Build complete: release/fintrack
@finlaydes →/workspaces/A02_financial_tracker (main) $ release/fintrack
Welcome to your Personal Finance Tracker!
Would you like to resume your previous session? (y/n): y
Resuming from last session...
Previous transactions loaded.

--- Personal Finance Tracker Menu ---
1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit
-----
Current balance: $20.00
Budget status: Within budget
Select an option (1-5): 4

--- All Transactions ---
Description          | Amount      | Status
-----
1. test              | $20.00      | saved
-----

--- Personal Finance Tracker Menu ---
1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit
-----
Current balance: $20.00
Budget status: Within budget

```

Select an option (1-5): 1

--- Add Income ---

Description: test2

Amount: 200

--- All Transactions ---

Description	Amount	Status
1. test	\$20.00	saved

Enter position to insert (1-2 or 2 to add at end): 2

Income added successfully

--- Personal Finance Tracker Menu ---

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

Current balance: \$220.00

Budget status: Within budget

Select an option (1-5): 4

--- All Transactions ---

Description	Amount	Status
1. test	\$20.00	saved
2. test2	\$200.00	new

--- Personal Finance Tracker Menu ---

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

--- Personal Finance Tracker Menu ---

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

Current balance: \$220.00

Budget status: Within budget

Select an option (1-5): 2

--- Add Expense ---

Description: test3

Amount: 400

--- All Transactions ---

Description	Amount	Status
1. test	\$20.00	saved
2. test2	\$200.00	new

Enter position to insert (1-3): 1

Expense added successfully

--- Personal Finance Tracker Menu ---

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

Current balance: \$-180.00

Budget status: Over budget

Current balance: \$-180.00
Budget status: Over budget
Select an option (1-5): 4

--- All Transactions ---

Description	Amount	Status
1. test3	\$-400.00	--- d
2. test	\$20.00	saved
3. test2	\$200.00	new

--- Personal Finance Tracker Menu ---

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

Current balance: \$-180.00
Budget status: Over budget
Select an option (1-5):

--- Personal Finance Tracker Menu ---

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

Current balance: \$-180.00
Budget status: Over budget
Select an option (1-5): 3

--- All Transactions ---

Description	Amount	Status
1. test3	\$-400.00	--- d
2. test	\$20.00	saved
3. test2	\$200.00	new

Enter transaction number to delete: 2
Transaction deleted successfully!

--- Personal Finance Tracker Menu ---

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

Current balance: \$-200.00
Budget status: Over budget
Select an option (1-5): 4

--- All Transactions ---

Description	Amount	Status
1. test3	\$-400.00	--- d
2. test2	\$200.00	new

```
--- Personal Finance Tracker Menu ---
```

1. Add Income
2. Add Expense
3. Delete
4. Print
5. Exit

```
-----
```

```
Current balance: $-200.00
```

```
Budget status: Over budget
```

```
Select an option (1-5): 5
```

```
Transactions saved successfully!
```

```
Done. Exiting program.
```

```
@finlaydes →/workspaces/A02_financial_tracker (main) $
```

```
logs > ≡ transaction_log.txt
1      test|20.00|saved
2
```

```
logs > ≡ transaction_log.txt
1  || test3|-400.00|saved
2  || test2|200.00|saved
3
```